

AYDT Registration Admin Portal — Full Architecture Document

Purpose of this document: This is the **living reference** for the entire system. It describes what exists, how it works, and where everything lives. Consult this when you need to understand the system's structure, data model, workflows, or integrations. For security vulnerabilities, production readiness, and risk analysis, see [ARCHITECTURE_REPORT.md \(/ARCHITECTURE_REPORT.md\)](#).

Updated: 2026-03-22
Stack: Next.js 16 · React 19 · TypeScript · Tailwind CSS 4 · Supabase · PostgreSQL · Elavon EPG · Twilio · Resend

Executive Summary

AYDT is a production-grade registration and administrative portal for a dance studio. The application supports:

- Public registration flow for families/dancers with multi-step wizard
- Admin dashboard for semester/class/payment/family management
- Email broadcast system with template rendering, scheduling, and delivery tracking
- Payment integration with Elavon EPG (hosted checkout, stored cards/ACH, recurring installments)
- SMS notifications via Twilio (opt-in, verified phone, delivery tracking)
- Waitlist automation with token-based acceptance and capacity management
- Tuition engine with progressive discounts, special programs, and coupon system
- Family account credits system for refunds and adjustments
- Media library for image and PDF management

As of March 22, 2026, the project has completed Phases 1–4 (class catalog, pricing, tuition, EPG payment) and Phase 5 (SMS — code-complete, awaiting Twilio A2P carrier approval).

1. Project Structure

```
aydt-registration-admin-portal/
├── app/
│   ├── (user-facing)/                # Public registration portal (route group)
│   │   ├── audition/[token]/        # Audition acceptance flow
│   │   ├── cart/                    # Shopping cart view
│   │   ├── error/                   # Error page
│   │   ├── family/                  # Family dashboard
│   │   ├── profile/                 # User profile (inline editing, SMS opt-in)
│   │   │   └── actions/              # addDancer, updateDancer, SMS actions
│   │   ├── register/               # Multi-step registration wizard
│   │   │   ├── layout.tsx           # CartRestoreGuard + progress indicator
│   │   │   ├── page.tsx            # Step router
│   │   │   └── confirmation/        # Final success page
```

- └── form/ # Dynamic form step
- └── participants/ # Dancer assignment step
- └── payment/ # EPG hosted checkout step
- └── semester/[id]/ # Semester detail + session browser
- └── waitlist/accept/[token]/ # Waitlist acceptance flow
- └── ConditionalFooter.tsx # Portal footer
- └── portal.css # Portal-specific styling (600+ lines)
- └── layout.tsx # Portal shell + nav + auth status
- └── admin/ # Admin management portal
 - └── _components/ # Admin layout components
 - └── TopBar.tsx # Sticky header with page title + actions
 - └── NotificationBell.tsx # Notification status indicator
 - └── DashboardRightPanel.tsx
 - └── EmailsRightPanel.tsx
 - └── EmailsTabSection.tsx # Email management UI (63 KB)
 - └── PaymentsRightPanel.tsx
 - └── SessionsRightPanel.tsx
 - └── layout.tsx # Admin shell + auth guard + sidebar nav
 - └── page.tsx # Dashboard landing (registrations, metrics, people tabs)
 - └── classes/ # Class management (CRUD)
 - └── [id]/ # Class detail view
 - └── semesters/ # Semester editor (9-step wizard)
 - └── [id]/ # Semester detail/tabs
 - └── dashboard/ # Semester stats dashboard
 - └── edit/ # Edit page (loads SemesterForm)
 - └── invites/ # Competition track invite management
 - └── new/ # New semester creation
 - └── discounts/ # Discount CRUD helpers
 - └── actions/ # Server actions (all semester mutations)
 - └── steps/ # Form step components
 - └── SemesterForm.tsx # Main editor orchestrator (useReducer)
 - └── emails/ # Email broadcast management
 - └── [id]/edit/ # Email edit page
 - └── new/ # New email draft
 - └── steps/ # Email wizard steps
 - └── actions/ # Email server actions
 - └── EmailForm.tsx # Email editor
 - └── families/ # Family records management
 - └── [id]/ # Family detail page
 - └── _components/ # FamilyDetailClient, ParentCard, DancerCard, panels
 - └── _components/ # Family list: modals, form helpers
 - └── actions/ # sendFamilyEmail
 - └── dancers/ # Dancer records management
 - └── users/ # User/admin management
 - └── payments/ # Payment tracking + admin adjustments
 - └── actions/ # markInstallmentPaid
 - └── sessions/ # Session management
 - └── media/ # Media library (images, PDFs, folders)
 - └── credits/ # Family account credits
 - └── actions/ # issueAccountCredit, issueBulkCredits
 - └── profile/ # Admin profile settings

- └── register/ # Admin manual registration tool
- └── api/ # API routes (JSON)
 - └── webhooks/
 - └── epq/route.ts # EPG payment webhook (625 lines)
 - └── resend/route.ts # Resend email delivery tracking
 - └── twilio/route.ts # SMS delivery status webhook
 - └── media/ # Media CRUD endpoints
 - └── route.ts # List/search images
 - └── [id]/route.ts # Update/delete image
 - └── folders/ # Folder CRUD
 - └── register/batch-status/ # Batch status polling endpoint
 - └── upload-image/route.ts # Image upload (5MB, MIME validation)
 - └── upload-pdf/route.ts # PDF upload (10MB)
- └── auth/ # Authentication flows
 - └── page.tsx # Auth landing (tabbed login/signup)
 - └── login/page.tsx
 - └── actions.ts # signUp, login, resetPassword server actions
 - └── confirm/route.ts
 - └── request-password-reset/
 - └── reset-password/
- └── components/ # Shared React components
 - └── public/ # ClassCard, SessionGrid, FilterBar, NavCartButton
 - └── semester-flow/ # CustomQuestionModal
 - └── semester-flow/ # TipTap editors, form builders
 - └── form/ # Inputs, selects, etc.
 - └── email/ # Email preview frame
 - └── media/ # Image picker modal
 - └── ui/ # Generic UI primitives
- └── providers/ # React Context providers
 - └── AuthProvider.tsx
 - └── CartProvider.tsx # Shopping cart (localStorage, 2hr TTL)
 - └── RegistrationProvider.tsx # Multi-step wizard state (sessionStorage)
 - └── SemesterDataProvider.tsx # Semester data hydration
- └── lib/
 - └── actions/ # Shared server action utilities
 - └── validation/ # Zod validation schemas
- └── lib/
 - └── schemas/registration.ts # Zod schemas for registration
- └── types/
 - └── index.ts # ALL domain types (~1827 lines)
 - └── public.ts # Public-safe type subset (~228 lines)
- └── utils/
 - └── supabase/
 - └── client.ts # Browser Supabase client (anon key)

- server.ts # Server Supabase client (anon key + cookies)
 - admin.ts # Admin/service role client (BYPASSES RLS)
 - middleware.ts # Auth session middleware
- payment/
 - epg.ts # EPG REST API client (Elavon, ~420 lines)
- email-templates/
 - subscriptionConfirmation.ts
- tuitionEngine.ts # Pricing calculation engine (~250 lines)
- buildPaymentSchedule.ts # Payment plan calculation
- detectTimeConflicts.ts # Registration conflict checking
- ratelimit.ts # Upstash rate limiting
- requireAdmin.ts # Admin auth guard (server-side)
- resolveEmailVariables.ts # Token replacement (canonical)
- prepareEmailHtml.ts # Email HTML normalization (Outlook)
- sendSms.ts # Twilio SMS integration (E.164, best-effort)
- gradeLabel.ts # Grade label mapping
- queries/
 - admin/
 - index.ts # Admin data queries (~500 lines)
 - getFamilyDetail.ts # Family detail query
- supabase/
 - migrations/ # 42+ migration files
 - 20260302000000_baseline.sql (3123 lines — core schema)
 - 20260304000000_session_pricing.sql
 - 20260304030000_hybrid_pricing.sql
 - 20260309000002_class_catalog.sql
 - 20260309000003_tuition_engine.sql
 - 20260312000002_discount_coupons.sql
 - 20260313000001_sms_opt_in.sql
 - 20260316000001_family_account_credits.sql
 - 20260316000002_epg_stored_payment.sql
 - 20260316000003_installment_charge_tracking.sql
 - 20260316000004_batch_form_data.sql
 - functions/ # Deno edge functions
 - process-waitlist/
 - process-scheduled-emails/
 - send-email-broadcast/
 - process-overdue-payments/ # Auto-charge installments
- scripts/
 - seed.ts # Base seed (auth users, forms, data)
 - seed-spring-2026-north.ts # North location seed
 - seed-spring-2026-ues.ts # UES location seed
- docs/ # Project documentation
 - ARCHITECTURE_FULL.md # This file
 - ARCHITECTURE_REPORT.md # Security + vulnerability audit
 - DESIGN_SYSTEM.md
 - EMAIL_SYSTEM.md
 - CLASS_CATALOG.md

		— CLASS_SESSION_ARCHITECTURE.md	
		— PRICING_AND_DISCOUNTS.md	
		— EPG_PAYMENT_INTEGRATION.md	
		— EPG_WEBHOOK_ARCHITECTURE.md	
		— SMS_NOTIFICATIONS_ARCHITECTURE.md	
		— RATE_LIMITING.md	
		— TESTING.md	
		— elavon/	# Elavon REST API specs
		— middleware.ts	# Auth + rate limiting + security headers
		— next.config.ts	
		— tailwind.config.js	
		— package.json	
		— tsconfig.json	

2. Dependencies (package.json)

Runtime Dependencies

Package	Version	Purpose
next	16.0.10	App framework
react / react-dom	19.2.0	UI library
@supabase/ssr	^0.7.0	Supabase SSR auth
@supabase/supabase-js	^2.80.0	Supabase client
tailwindcss	^4	CSS framework
zod	^4.1.13	Schema validation
react-hook-form	^7.67.0	Form state management
@hookform/resolvers	^5.2.2	Zod + RHF bridge
@tiptap/react	^3.20.0	Rich text editor
@tiptap/starter-kit	^3.20.0	TipTap base extensions
resend	^6.9.2	Transactional email delivery
recharts	^3.8.0	Dashboard charts
lucide-react	^0.577.0	Icons
uuid	^13.0.0	UUID generation
@upstash/ratelimit	^2.0.8	Rate limiting
@upstash/redis	^1.37.0	Redis client
twilio	^5.13.0	SMS
image-size	^2.0.2	Image dimension detection
sass	^1.94.2	SCSS support

Dev Dependencies

Package	Purpose
typescript ^5	Type checking
eslint ^9	Linting
vitest ^4.0.14	Unit tests
playwright ^1.57.0	E2E tests

@testing-library/react ^16 Component testing

Scripts

```
npm run dev      # Start dev server
npm run build    # Build for production
npm run start    # Run production server
npm run type-check # TypeScript check
npm run lint     # ESLint
npm run test     # Vitest (unit tests)
npm run test:e2e # Playwright (e2e tests)
npm run db:new   # Create new migration
npm run db:seed  # Seed database
```

3. Core Domain Types (types/index.ts)

User

```
interface User {
  id: string
  family_id: string
  email: string
  first_name: string
  last_name: string
  phone_number: string
  is_primary_parent: boolean
  role: 'super_admin' | 'admin' | 'parent'
  status: string
  sms_opt_in?: boolean
  sms_verified?: boolean
  address_line1?: string | null
  city?: string | null
  state?: string | null
  zipcode?: string | null
  signature_html?: string | null
  signature_config?: SignatureConfig | null
}
```

Family

```
interface Family {
  id: string
  family_name: string | null // NOTE: column is family_name, NOT name
  created_at: string
  users: User[]
  dancers: Dancer[]
}
```

Semester

```
interface Semester {  
  id: string  
  name: string  
  status: 'draft' | 'scheduled' | 'published' | 'archived'  
  registration_form: RegistrationFormElement[] // JSONB  
  confirmation_email: ClassEmailConfig // JSONB  
  waitlist_settings: WaitlistSettings // JSONB  
  payment_plan?: PaymentPlan  
  publish_at?: string | null  
  published_at?: string | null  
}
```

Registration & Payment

```

interface RegistrationBatch {
  id: string
  parent_id?: string
  semester_id: string
  family_id: string
  cart_snapshot: CartSnapshot[]
  form_data_snapshot: Record<string, unknown>
  payment_plan_type?: string
  grand_total?: number
  amount_due_now?: number
  stored_payment_method_id?: string // FK to stored_payment_methods
  status: 'pending' | 'confirmed' | 'failed' | 'refunded'
}

interface Payment {
  id: string
  registration_batch_id: string
  order_id?: string // EPG order ID
  payment_session_id?: string // EPG session ID
  transaction_id?: string // EPG transaction ID
  custom_reference: string // batchId (for reconciliation)
  amount: number
  state: 'initiated' | 'pending_authorization' | 'authorized' | 'captured'
    | 'settled' | 'declined' | 'voided' | 'refunded' | 'held_for_review'
  raw_notification?: Record<string, unknown> // EPG webhook payload (audit)
  raw_transaction?: Record<string, unknown> // Full EPG response (audit)
}

interface BatchPaymentInstallment {
  id: string
  registration_batch_id: string
  installment_number: number
  amount: number
  due_date: string
  status: 'pending' | 'scheduled' | 'charged' | 'paid' | 'overdue' | 'failed' | 'waived'
  charge_attempt_count: number // max 3
  last_charge_error?: string
  transaction_id?: string // EPG txn ID from auto-charge
}

```

EPG Stored Payment


```

interface EpgStoredCard {
  id: string
  shopper: string // Parent shopper HREF
  card?: { maskedNumber, last4, scheme, expirationMonth, expirationYear }
}

interface EpgStoredAchPayment {
  id: string
  shopper: string
  last4?: string
  achAccountType?: string
  accountName?: string
}

```

Family Account Credits

```

interface FamilyAccountCredit {
  id: string
  family_id: string
  amount: number
  reason?: string | null
  issued_by_admin_id?: string
  source_batch_id?: string // Batch that triggered the credit
  used_in_batch_id?: string // Batch that consumed the credit
  is_active: boolean
}

```

4. Database Schema (Supabase PostgreSQL — 42+ migrations)

Core Tables

Table	Purpose
users	Auth + profile + address + SMS opt-in; role = super_admin / admin / parent
families	Family grouping with family_name column
dancers	Student records, linked to users via join table
semesters	Enrollment periods (draft → published → archived); JSONB: registration_form, confirmation_email, waitlist_settings
classes	Curriculum units (discipline, division, visibility, enrollment_type)
class_sessions	Time slots (day, time, capacity, instructor, dates)
session_occurrence_dates	Individual calendar dates for absence tracking
session_groups	Bundled class offerings
registrations	Student enrollments (30-min hold, status = pending/confirmed/cancelled)
registration_batches	Multi-registrant checkout with cart_snapshot + form_data_snapshot JSONB
batch_payment_installments	Auto-pay schedule (installment_number, amount, due_date, status, charge tracking)
payments	EPG payment state + raw_notification + raw_transaction JSONB
installment_charges	Per-charge tracking for recurring auto-pay attempts
shoppers	EPG customer records (linked to user_id)

stored_payment_methods	Card/ACH records (type discriminator, masked data only)
family_account_credits	Dollar credits issued by admin; linked to source/used batches
tuition_rate_bands	Progressive pricing by division + weekly class count
special_program_tuition	Fixed fees for technique/pointe/competition/early_childhood
semester_fee_config	Per-semester fee constants
waitlist_entries	Capacity overflow management with token-based invite
emails	Broadcast email records with TipTap JSON body
email_recipients	Snapshotted recipient list at send time
email_deliveries	Per-recipient delivery tracking via Resend webhooks
email_subscriptions	Opt-in/opt-out tracking
sms_notifications	SMS delivery records (twilio_sid, status, error_message)
discounts / semester_discounts	Global discount definitions linked to semesters
media_images	Image library with folder organization
media_folders	Folder structure for media organization
semester_audit_logs	Admin action history

Key Indexes

```

registrations(session_id, dancer_id, status)
registrations(registration_batch_id)
batch_payment_installments(registration_batch_id, status)
waitlist_entries(session_id, status)
session_occurrence_dates(session_id, date)
email_recipients(email_id, user_id)
email_deliveries(email_id, user_id, status)
stored_payment_methods(shopper_id) WHERE stored_payment_method_id IS NOT NULL

```

Database Functions & Triggers

```

-- Auth
handle_new_user()           -- Auto-creates family + user on Supabase signup
is_admin_or_super()         -- RLS helper: role IN ('admin','super_admin')
is_super_admin()            -- RLS helper: role = 'super_admin'
is_instructor()             -- RLS helper: role = 'instructor'

-- Registration
check_registration_capacity() -- Enforces session capacity at DB level
check_registration_time_conflict() -- Prevents overlapping session times
prevent_child_modification_if_semester_published()
prevent_semester_core_edit_if_published()

-- Cron Jobs (pg_cron)
expire_registration_holds_cron() -- Releases 30-min pending holds
process_waitlist()              -- Invites next in line (every 1 minute)
process_scheduled_emails()      -- Dispatches scheduled sends

```

5. Authentication & Authorization

Auth Flow (Supabase)

1. **Signup:** Email/password → Supabase Auth → `handle_new_user()` trigger creates users + families row
2. **Login:** Email + password → Supabase Auth → JWT in `httpOnly` cookie (via `updateSession()` in middleware)
3. **Middleware Guard:** `utils/supabase/middleware.ts` server-side check for `/admin` paths → redirects unauthenticated or non-admin users before page renders
4. **Client Guard:** `app/admin/layout.tsx` defense-in-depth check for role IN ['admin', 'super_admin']
5. **Server-side Guard:** `utils/requireAdmin.ts` verifies role IN ['admin', 'super_admin'] for server actions + API routes

Roles

Role	Access
super_admin	Full admin portal access
admin	Full admin portal access
instructor	Read-only access to assigned classes/sessions (via <code>class_sessions.instructor_id</code>)
parent	User-facing portal only

Supabase Clients

```
// utils/supabase/server.ts — for server components + server actions
createClient() // Uses anon key + cookie auth → RLS ENFORCED

// utils/supabase/admin.ts — for privileged server actions + webhooks
createAdminClient() // Uses service role key → BYPASSES RLS

// utils/supabase/client.ts — for client components
createBrowserClient() // Uses anon key → RLS ENFORCED
```

Row-Level Security (RLS)

39 tables have RLS enabled with role-based policies (as of migration 20260323000001). Key patterns:

- **Admins** (`is_admin_or_super()`): full CRUD on all admin-managed tables
- **Parents:** read own family/payment/installment data (scoped via `registration_batches.parent_id` or `dancers.family_id`), read published catalog (classes, sessions, fees, requirements)
- **Instructors** (`is_instructor()`): read-only on classes/sessions/occurrence dates where `class_sessions.instructor_id = auth.uid()`
- **Service role:** full bypass (used in webhooks, cron jobs, edge functions)
- **Public:** read published semesters + sessions + discounts + fee config + tuition bands

Previously 9 tables lacked RLS (`payments`, `batch_payment_installments`, `class_sessions`, `classes`, `class_requirements`, `requirement_waivers`, `semester_fee_config`, `session_occurrence_dates`, `tuition_rate_bands`). All were secured in the March 23 migration.

6. API Routes & Endpoints

User-Facing Routes

GET /semester/[id]	— Semester detail + session browser
GET /register	— Multi-step registration wizard
GET /register/form	— Dynamic form step
GET /register/participants	— Dancer assignment step
GET /register/payment	— EPG hosted checkout redirect
GET /register/confirmation	— Success page
GET /waitlist/accept/[token]	— Token-based waitlist acceptance
GET /audition/[token]	— Audition/invite acceptance
GET /family	— Family dashboard
GET /profile	— User profile (inline editing + SMS)

Admin Routes

GET /admin	— Dashboard landing (registrations, metrics, people)
GET /admin/semesters	— Semester listing
GET /admin/semesters/new	— New semester creation (9-step wizard)
GET /admin/semesters/[id]/edit	— Semester editor
GET /admin/semesters/[id]/dashboard	— Semester stats
GET /admin/classes	— Class listing
GET /admin/families	— Family listing
GET /admin/families/[id]	— Family detail (parents, dancers, inline editing)
GET /admin/payments	— Payment tracking dashboard
GET /admin/emails	— Email broadcast listing + analytics
GET /admin/credits	— Family account credits management
GET /admin/media	— Media library (images, PDFs, folders)
GET /admin/sessions	— Session management

JSON API Endpoints

GET /api/register/batch-status	— Poll registration batch status (auth + ownership)
GET /api/media	— List media images (admin auth required)
POST /api/media	— Create media record (admin auth required)
PATCH /api/media/[id]	— Update media metadata (admin auth required)
DELETE /api/media/[id]	— Delete media (admin auth required)
GET /api/media/folders	— List folders (admin auth required)
POST /api/media/folders	— Create folder (admin auth required)
DELETE /api/media/folders/[name]	— Delete folder (admin auth required)
POST /api/upload-image	— Image upload, 5MB, MIME validated (admin auth required)
POST /api/upload-pdf	— PDF upload, 10MB (admin auth required)
POST /api/webhooks/epg	— EPG payment webhook (HTTP Basic auth, mandatory)
POST /api/webhooks/resend	— Resend delivery tracking (Svix signature, mandatory)
POST /api/webhooks/twilio	— Twilio SMS callback (Twilio signature, mandatory)

7. Middleware (middleware.ts + utils/supabase/middleware.ts + utils/ratelimit.ts)

Request Flow

1. **Rate limiting** — per-IP via Upstash Redis sliding window:

```
/api/webhooks → 100 req/60s  
/api/upload-* → 10 req/60s  
/api/media → 60 req/60s  
/api/register → 30 req/60s
```

2. **Session refresh** — `updateSession()` creates Supabase client, refreshes auth cookies
3. **Admin route protection** — for `/admin` paths:
 - No user → redirect to `/auth`
 - User without admin or `super_admin` role → redirect to `/`
4. **Security headers** — HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, X-XSS-Protection

8. State Management

Context Providers

Provider	Storage	Responsibilities
AuthProvider	Memory	Supabase auth state, user info, sign-out
CartProvider	localStorage (2hr TTL)	Cart items, <code>addItem()</code> , <code>removeItem()</code> , <code>clear()</code>
RegistrationProvider	sessionStorage	Multi-step wizard: email, participants, form data, batch ID, fingerprint
SemesterDataProvider	Memory (hydrated)	Semester + sessions data during browsing

Semester Editor State (SemesterForm.tsx)

```
const [state, dispatch] = useReducer(semesterReducer, initialState)  
  
// Dirty tracking — warns on navigation if unsaved  
useEffect(() => {  
  if (isDirty) window.addEventListener('beforeunload', handleBeforeUnload)  
}, [isDirty])  
  
// On step change:  
await persistSemesterDraft(state) // Full sync of sessions, groups, plans, discounts  
navigate(nextStep)
```

9. Key Utilities

Tuition Engine (utils/tuitionEngine.ts)

Centralized pricing calculation supporting:

- **Progressive discounts:** Per additional weekly class (e.g., 2nd class 5% off base)
- **Special programs:** Fixed fees (Technique, Pointe, Competition, Early Childhood)
- **Weekly class limits:** Junior max 3/week, Senior max 6/week
- **Costume fees:** Senior \$65/class, Junior \$55/class (exempt: technique, pointe, competition)
- **Registration fee:** \$40/child

- **Video fees:** \$15/registrant (senior only)
- **Family discount:** \$50 off when 2+ dancers register

Key functions: `validateWeeklyLimit()`, `isSpecialProgramClass()`, `getSpecialProgramKey()`, `calculateClassTuition()`

EPG API Client (`utils/payment/epg.ts`)

HTTP Basic auth against Elavon REST API (~420 lines, server-only):

```
createEpgOrder()           // Create order with amount + batchId
createEpgPaymentSession()  // Get hosted checkout URL (doCapture, doThreeDSecure)
fetchEpgPaymentSession()   // Re-fetch existing session (idempotency)
fetchEpgTransaction()      // Query authoritative transaction state
createEpgShopper()         // Create customer profile (linked to user_id)
fetchEpgShopperByReference() // Find existing shopper
createEpgStoredCard()       // Save card from hostedCard token
createEpgStoredAchPayment() // Save ACH from hostedAchPayment token
createEpgTransaction()      // Server-to-server charge (installments)
epgEventTypeToPaymentState() // Map EPG events → internal states
```

SMS (`utils/sendSms.ts`)

```
sendSms(to, body) // E.164 normalization, best-effort delivery, logs to sms_notifications
```

- Never throws — failures logged but don't block calling flows
- Automatic logging to `sms_notifications` table with Twilio SID

Email Token Replacement (`utils/resolveEmailVariables.ts`)

```
resolveEmailVariables(htmlBody, {
  parent_name, student_name, semester_name, session_name,
  hold_until_datetime, accept_link, unsubscribe_link, ...
})
// Unknown tokens preserved as {{key}} — safe for forward compatibility
```

10. Major Workflows

User Registration Flow

1. **User** visits /semester/[id]
 - └─ **Server** fetches semester + sessions (**with** daysOfWeek)
 - └─ Renders ClassCard grid **with** CartDrawer
2. **User** adds sessions **to** cart
 - └─ CartProvider updates localStorage
 - └─ TTL: now + 2 hours
3. **User** clicks Checkout → /register
 - └─ Step 1: Email entry (auth lookup / magic link)
 - └─ Step 2: Participants (**select/create** dancers, assign **to** sessions)
 - └─ Custom questions rendered per **session**
 - └─ Step 3: Dynamic form (semester-specific registration form **JSONB**)
 - └─ Step 4: Payment
 - └─ Pricing computed **server**-side (tuition engine)
 - └─ Coupon validation (validateCoupon **server** action)
 - └─ Payment plan selection (**full**-pay vs installments)
 - └─ RegistrationBatch created (status=**pending**)
 - └─ EPG **order** + payment **session** created
 - └─ **User** redirected **to** EPG hosted checkout page
 - └─ Step 5: Confirmation
4. EPG webhook (**on** payment success)
 - └─ **Validate** HTTP Basic auth (**timing**-safe)
 - └─ **Fetch** authoritative **transaction** from EPG
 - └─ **Update** Payment **record** state
 - └─ **Full**-pay: confirmBatch() immediately
 - └─ Installments: store card → charge installment 1 → confirmBatch()
 - └─ confirmBatch():
 - └─ batch.status = **'confirmed'**
 - └─ Registration **rows** created (1 per dancer-**session** pair)
 - └─ Installment 1 marked paid
 - └─ Send confirmation email

EPG Payment Processing (Full-Pay vs Installments)

FULL-PAY (doCapture: **true**)

EPG HPP charges immediately → webhook → confirmBatch()

INSTALLMENTS (doCapture: **false**)

EPG HPP authorizes (**no** charge) → webhook (saleAuthorized)

→ storePaymentMethodAndCaptureInstallment():

1. **Fetch** hostedCard/hostedAchPayment token **from session**
2. **Create or** find EPG Shopper (linked **to** user_id)
3. **Create** Stored Card **or** Stored ACH **from** token
4. Charge installment **1 server-to-server**
 - └─ **If** authorized: confirmBatch() + mark installment **1** paid
 - └─ **If** declined: **log** error, leave batch pending (**admin** retry)
5. Link stored **method to** batch **for** future charges

AUTO-PAY (process-overdue-payments edge **function**, daily cron)

1. Find installments **WHERE** status=**'scheduled'** **AND** due_date < today
2. Mark **as** **'overdue'**
3. Charge via stored payment **method** (up **to** **3** attempts)
4. **On** success: mark **'paid'**, send receipt email + SMS
5. **On** final failure: mark **'failed'**, alert **admin**
6. Send **admin summary** email

Email Broadcast

1. **Admin** creates email (status=**'draft'**)
 - └─ Fills subject, sender, body (TipTap)
 - └─ Selects recipients: semester | **session** | manual
2. **Admin** schedules **or** sends immediately
 - └─ **Snapshot** recipients → email_recipients **table**
 - └─ status = **'scheduled'** | **'sending'**
3. process-scheduled-emails edge fn (pg_cron every **1**min)
 - └─ Atomically claim: status → **'sending'** (prevents **double**-send)
 - └─ Invoke send-email-broadcast **for each** email
 - └─ Mark **'sent'** **only AFTER** broadcast confirms completion
4. send-email-broadcast edge fn
 - └─ Process **in** batches **of** **100** (Promise.allSettled)
 - └─ resolveEmailVariables() **for each** recipient
 - └─ prepareEmailHtml() **for** Outlook compatibility
 - └─ resend.emails.send() **for** delivery
 - └─ Upsert email_deliveries **with** resend_message_id
5. Resend webhook → /api/webhooks/resend
 - └─ Updates email_deliveries: delivered, bounced, opened, clicked

Waitlist Automation

pg_cron **every 1 minute** → process-waitlist edge **function**

Step 1: Expire seat holds (registrations **WHERE** status='pending' **AND** hold_expires_at < now)

Step 2: Expire stale invites (waitlist_entries **WHERE** status='invited' **AND** expired)

Step 3: **For each** session **with** capacity:

- └─ **One-at-a-time** rule: **no** active 'invited' entry
- └─ **Check** available capacity
- └─ **Fetch** next waiting entry (**ORDER BY** position)
- └─ Update status → 'invited'
- └─ Send invitation email (token link)
- └─ Send SMS if opted **in**

User visits /waitlist/accept/[token]

- └─ Validate token (**not** expired, status='invited')
- └─ **Create** registration (status='pending', 30-min hold)
- └─ Update entry → 'accepted'

SMS Notifications

Event triggers (waitlist invite, overdue payment, etc.)

- └─ Check user.sms_opt_in AND user.sms_verified
- └─ sendSms(phone, message) — best-effort, never throws
- └─ Log to sms_notifications table
- └─ Twilio webhook → /api/webhooks/twilio updates delivery status

User SMS Setup (profile page)

- └─ Toggle SMS opt-in
- └─ Enter phone number (E.164 normalized)
- └─ Receive verification code
- └─ Confirm code → mark sms_verified

11. Environment Variables

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

# Email (Resend)
RESEND_API_KEY=
RESEND_FROM_EMAIL=
RESEND_WEBHOOK_SECRET=      # Svix signature verification

# Payment (Elavon EPG)
EPG_BASE_URL=                # UAT or production
EPG_MERCHANT_ALIAS=
EPG_SECRET_KEY=
EPG_WEBHOOK_USERNAME=
EPG_WEBHOOK_PASSWORD=

# SMS (Twilio)
TWILIO_ACCOUNT_SID=
TWILIO_AUTH_TOKEN=
TWILIO_PHONE_NUMBER=
TWILIO_WEBHOOK_AUTH_TOKEN=   # Webhook signature validation

# Rate limiting (Upstash)
UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=

# Site
NEXT_PUBLIC_BASE_URL=
SITE_URL=
ADMIN_NOTIFICATION_EMAIL=    # For auto-pay summaries
```

12. Key Architectural Patterns

Server Actions (Next.js App Router)

All mutations via server actions — no REST API for admin or registration:

```
"use server"

export async function persistSemesterDraft(state) { ... }
export async function publishSemester(id) { ... }
export async function sendEmailNow(emailId) { ... }
export async function markInstallmentPaid(installmentId) { ... }
export async function createEPGPaymentSession(batchId, amount) { ... }
export async function chargeStoredPaymentInstallment(installmentId) { ... }
export async function issueAccountCredit(familyId, amount, reason) { ... }
```

Form Management

- React Hook Form + Zod for registration steps

- **useReducer** for multi-step admin forms (SemesterForm, EmailForm)
- **TipTap** for rich text email body editing
- **Dynamic field builder** for semester registration forms (JSONB schema)
- **Inline editing** for profile cards (field-level, no modals)

Database Access Patterns

- **Server components** call Supabase directly (with `getClient()` — anon key, RLS enforced)
- **Server actions** use admin client (`createAdminClient()`) for privileged mutations
- **Client components** use Supabase browser client for real-time or user-scoped queries
- **No ORM** — raw Supabase query builder throughout

Admin UI Architecture

- **TopBar** + optional **Right Panels** for context-aware actions
- Collapsible sidebar navigation
- Right panels per section: `DashboardRightPanel`, `PaymentsRightPanel`, `EmailsRightPanel`, etc.
- Avatar color system (deterministic from name hash, 5-color palette)
- Status badges: consistent styling for enrollment, email, credit, payment states

13. Key Files Reference

File	Purpose	Lines
<code>types/index.ts</code>	All domain types — single source of truth	~1827
<code>types/public.ts</code>	Public-safe type subset	~228
<code>app/admin/layout.tsx</code>	Admin shell + client-side auth guard + sidebar nav	~212
<code>app/admin/semesters/SemesterForm.tsx</code>	Multi-step editor orchestrator	~500
<code>app/admin/page.tsx</code>	Dashboard landing page	~400
<code>utils/tuitionEngine.ts</code>	Pricing calculation engine	~250
<code>utils/payment/epg.ts</code>	EPG REST API client	~420
<code>utils/sendSms.ts</code>	Twilio SMS integration	~100
<code>utils/resolveEmailVariables.ts</code>	Token replacement utility	~100
<code>queries/admin/index.ts</code>	Admin data queries	~500
<code>queries/admin/getFamilyDetail.ts</code>	Family detail query	~100
<code>app/api/webhooks/epg/route.ts</code>	EPG payment webhook handler	~625
<code>middleware.ts</code>	Rate limiting + admin route guard + security headers	~61
<code>utils/supabase/middleware.ts</code>	Session refresh + server-side admin auth + security headers	~95
<code>supabase/migrations/20260323000001_*.sql</code>	RLS on 9 tables + instructor role + policies	~290
<code>supabase/migrations/20260302000000_baseline.sql</code>	Core schema	3123
<code>supabase/functions/process-waitlist/index.ts</code>	Waitlist automation cron	~300
<code>supabase/functions/send-email-broadcast/index.ts</code>	Batch email sender	~200
<code>supabase/functions/process-overdue-payments/index.ts</code>	Auto-charge installments	~487

14. Development Phases

Phase	Description	Status
1	Class catalog + session architecture	Complete
2	Semester pricing engine	Complete
3	Tuition engine + fee config	Complete
4	EPG payment integration (Elavon)	Complete (2026-03-16)
5	SMS notifications (Twilio)	Code-complete; blocked on A2P carrier approval
6	RLS policy hardening + instructor role	Complete (2026-03-23)
7	Competition program enhancements	Planned
8	Advanced reporting & analytics	Planned

15. Deployment

- **App:** Next.js 16 deployed to **Vercel**
 - **Database:** **Supabase** hosted PostgreSQL (with pg_cron for scheduled jobs)
 - **Edge Functions:** **Supabase Functions** (Deno runtime) for waitlist + email + payment crons
 - **Email:** **Resend** API for transactional + broadcast email
 - **Payments:** **Elavon EPG** (hosted checkout + stored cards/ACH + recurring)
 - **CDN:** Vercel Edge Network
 - **Redis:** **Upstash** for rate limiting
 - **SMS:** **Twilio** (pending A2P approval)
-

16. Integration Architecture

How EPG Works (for non-payment developers)

Elavon Payment Gateway (EPG) is the payment processor. The integration uses a **hosted payment page** (HPP) model, which means:

1. **Your server creates an "order"** with the amount and a reference ID (the batch ID).
2. **Your server creates a "payment session"** linked to that order. EPG returns a URL.
3. **The user is redirected to that URL** — an Elavon-hosted page where they enter their card or ACH info. Your application never sees or handles raw card numbers.
4. **Elavon processes the payment** and sends a webhook notification to your server.
5. **Your webhook handler** receives the notification, re-fetches the authoritative transaction state from Elavon (never trusting the webhook payload alone), and updates your database.

For **installment plans**, the HPP authorizes but doesn't charge (doCapture: false). Instead, it returns a one-time token that your server exchanges for a "stored card" or "stored ACH" — a persistent reference Elavon keeps. Future charges use this stored reference via server-to-server API calls, no user interaction needed.

How SMS Works (for non-Twilio developers)

Twilio handles SMS delivery. The integration:

1. **Sends SMS** via Twilio's API with the recipient's phone number and message body.
2. **Logs the attempt** in the sms_notifications table with the Twilio message SID.
3. **Receives status updates** via webhook when Twilio reports delivery, failure, or undelivery.

4. **SMS is always best-effort** — failures never block the calling workflow (payment confirmations, waitlist invites, etc.).

Users must **opt in** (toggle in profile) and **verify their phone** (receive a code) before receiving SMS.

Carrier compliance: Twilio requires A2P 10DLC brand + campaign registration (~\$39, ~1 week approval) before production SMS delivery. Code is complete; this is an external approval dependency.

How Email Broadcasting Works (for non-email developers)

Resend is the transactional email provider. Broadcasting works in stages:

1. **Admin composes** an email using a TipTap rich text editor with template variables (e.g., `{{parent_name}}`).
2. **Admin selects recipients** by semester, session, or manual list.
3. **On schedule/send**, recipients are **snapshotted** into a frozen list (preventing mid-send drift).
4. **A cron job** picks up scheduled emails, atomically claims them (status → "sending"), and invokes a Deno edge function.
5. **The edge function** processes recipients in batches of 100, replacing template variables and calling the Resend API.
6. **Resend webhooks** report delivery status back (delivered, bounced, opened, clicked) per recipient.

Updated: March 22, 2026

System: AYDT Registration Admin Portal

Document type: Living reference architecture